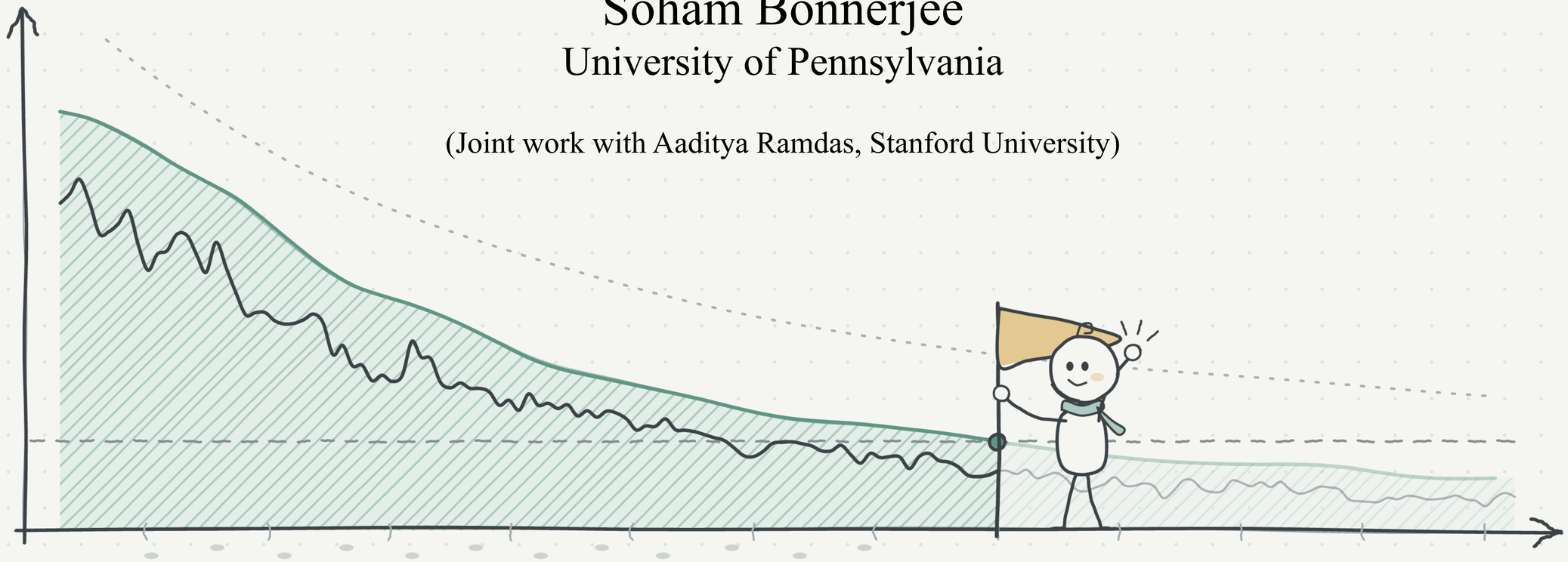
A general recipe of anytime-valid envelopes for iterative algorithms

Soham Bonnerjee
University of Pennsylvania

(Joint work with Aaditya Ramdas, Stanford University)

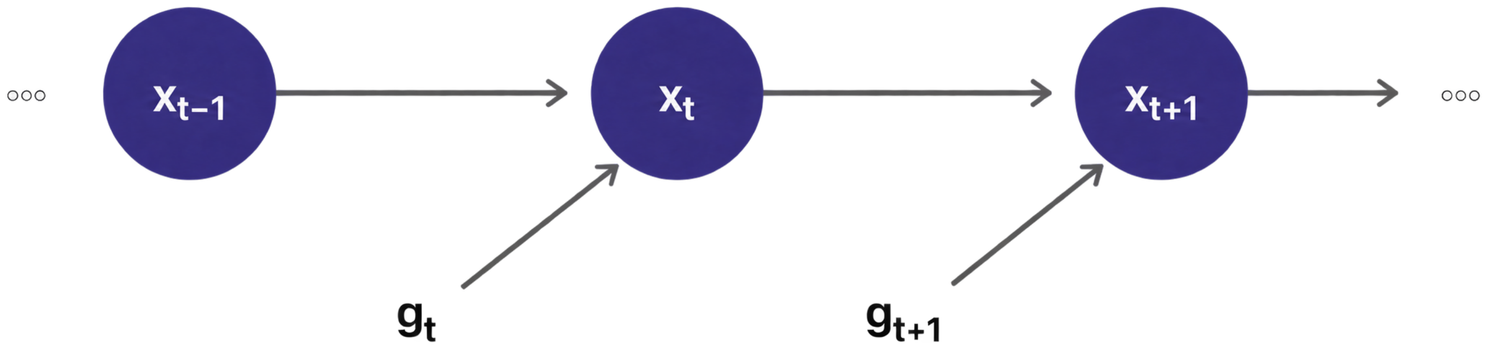


Agenda

- Iterates x_t; e.g. SGD: $x_t = \Pi_{\mathcal{X}}(x_{t-1} - \eta_t g_t)$ for $\min_{x \in \mathcal{X}} F(x)$.

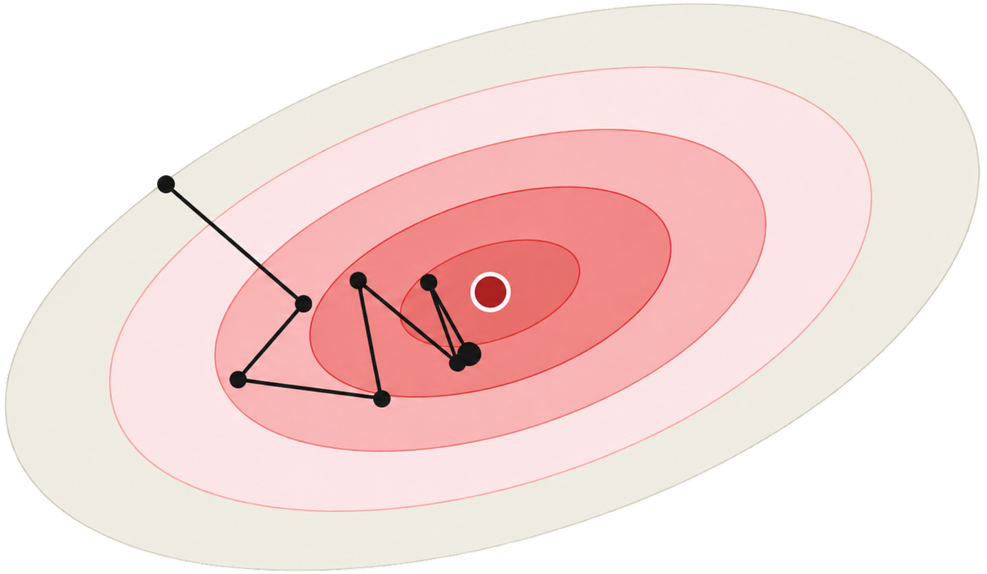
Step-size/learning rate

Stochastic gradients



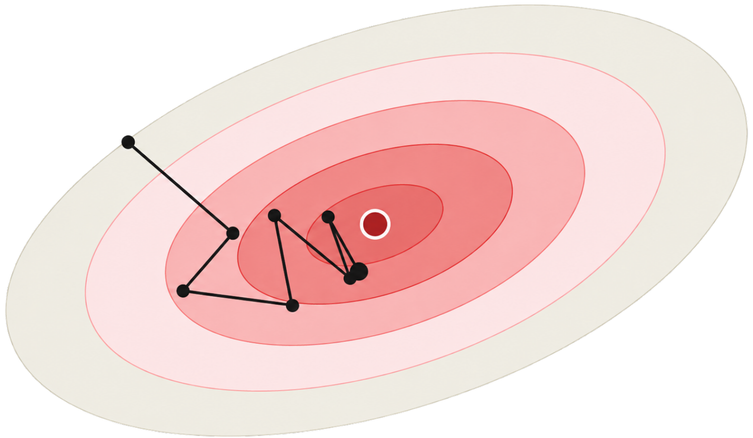
Agenda

- Iterates x_t; e.g. SGD: $x_t = \Pi_{\mathcal{X}}(x_{t-1} - \eta_t g_t)$ for $\min_{x \in \mathcal{X}} F(x)$.
- Goal: Time-uniform boundaries for some performance measure Z_t of x_t.



Agenda

- Iterates x_t; e.g. SGD: $x_t = \Pi_{\mathcal{X}}(x_{t-1} - \eta_t g_t)$ for $\min_{x \in \mathcal{X}} F(x)$.
- Goal: Time-uniform boundaries for some performance measure Z_t of x_t.



For SGD, e.g.

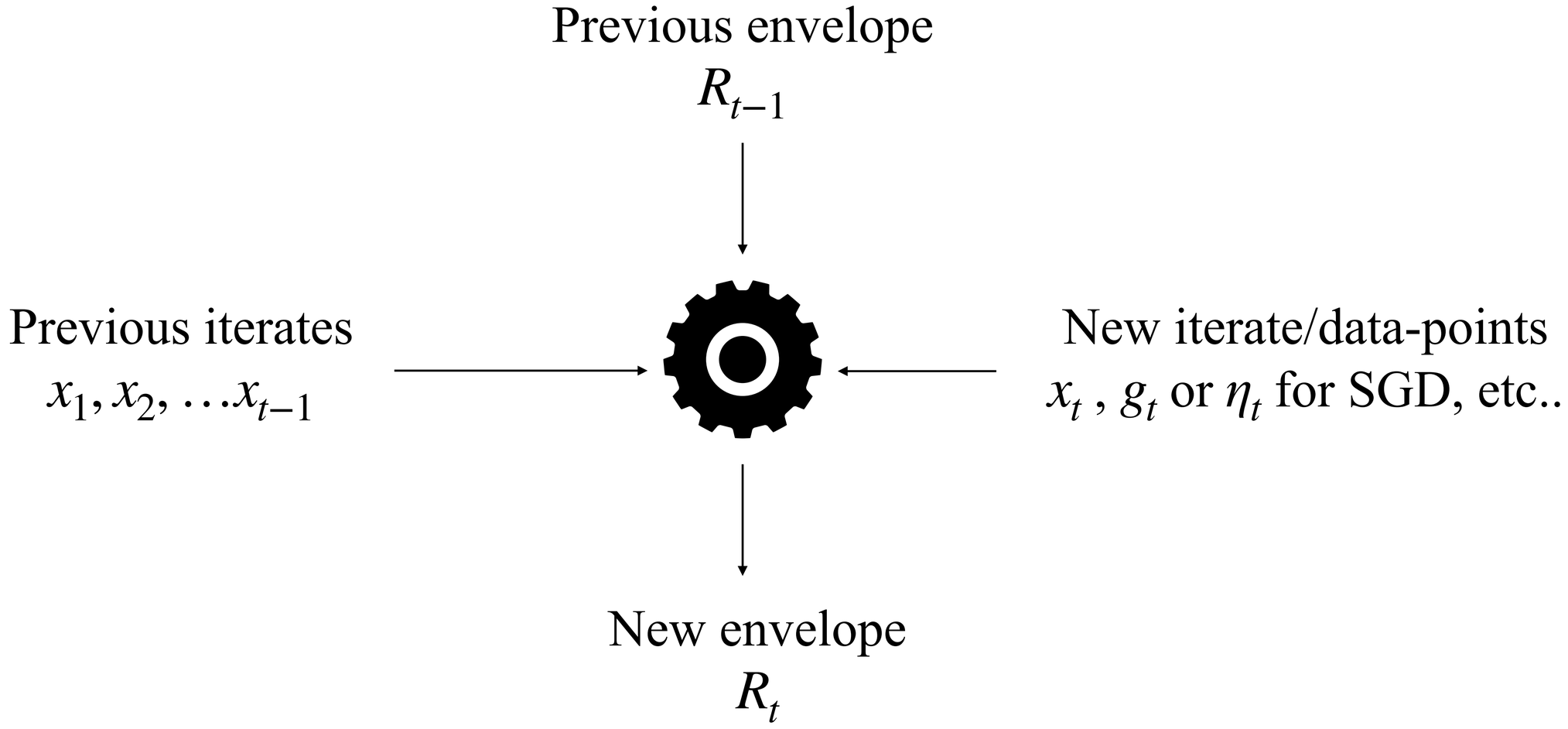
- $\|x_t - x^\star\|^2$, $x^\star = \arg\min_{x \in \mathcal{X}} F(x)$
- $F(x_t) - F^\star$, $F^\star = \min_{x \in \mathcal{X}} F(x)$
- $\|\nabla F(x_t)\|$

Agenda

- Iterates x_t; e.g. SGD: $x_t = \Pi_{\mathcal{X}}(x_{t-1} - \eta_t g_t)$ for $\min_{x \in \mathcal{X}} F(x)$.
- Goal: Time-uniform boundaries for some performance measure Z_t of x_t.

$$\mathbb{P}(Z_t \leq R_t \text{ for all } t \geq 1) \geq 1 - \alpha.$$

A generic envelope construction



Outline of this talk

- [ ] Why envelopes, and must we build them case by case?
- [ ] So what's new?- a modularized perspective.
- [ ] How to construct sharper envelopes?
- [ ] Application



Convex/ strongly convex/ L-smooth

Trajectory-adaptive, observable envelope

Interpretability



Convex/ strongly convex/ L-smooth

Trajectory-adaptive, observable envelope

Interpretability

Generalizability



Convex/ strongly convex/ L-smooth

Trajectory-adaptive, observable envelope

Interpretability

Generalizability

When to stop an iterative algorithm?

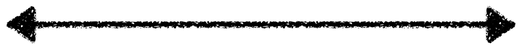
Other assumptions? Beyond SGD?

Desiderata 1

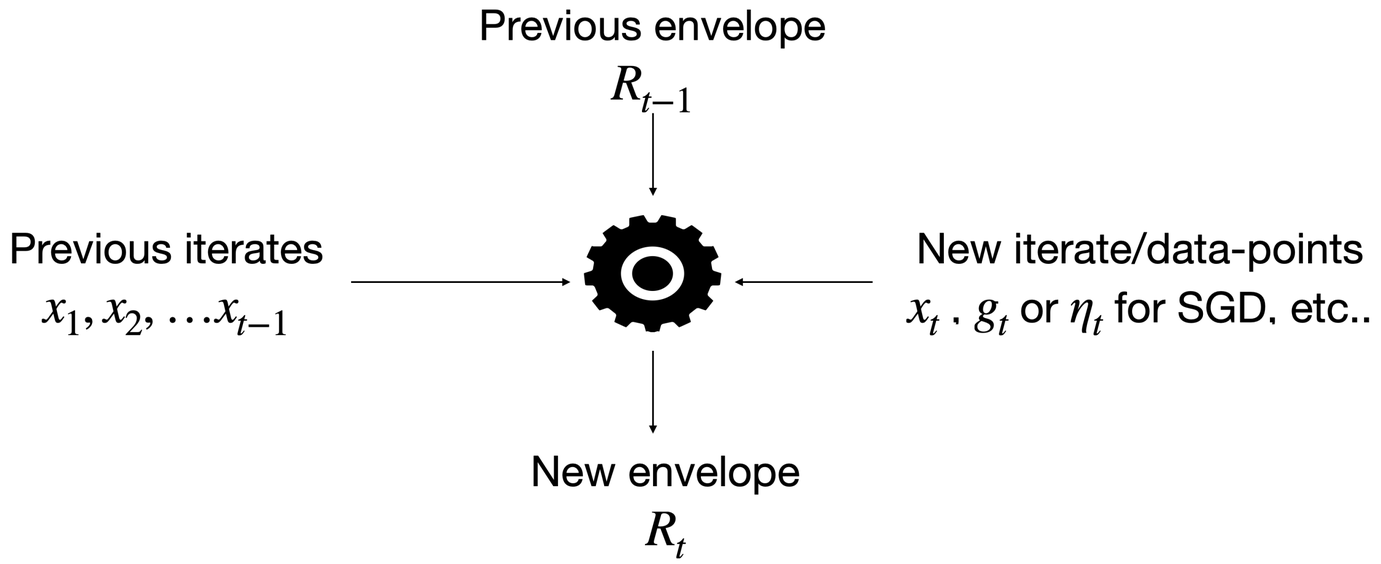
When to stop an iterative algorithm?

Samples t such that envelope $R_t < \varepsilon$

Cost of envelope construction



Physical cost of sampling or gradient computation



When to stop an iterative algorithm?

Physical cost of sampling
or
gradient computation

Neural Architecture Search with Reinforcement Learning

Barret Zoph, Quoc Le

Published: 06 Feb 2017, Last Modified: 12 Oct 2025 ICLR 2017 Oral Readers: Everyone Show Bibtex Show Revisions

Abstract: Neural networks are powerful and flexible models that work well for many difficult learning tasks in image, speech and natural language understanding. Despite their success, neural networks are still hard to design. In this paper, we use a recurrent network to generate the model descriptions of neural networks and train this RNN with reinforcement learning to maximize the expected accuracy of the generated architectures on a validation set. On the CIFAR-10 dataset, our method, starting from scratch, can design a novel network architecture that rivals the best human-invented architecture in terms of test set accuracy. Our CIFAR-10 model achieves a test error rate of 3.65, which is 0.09 percent better and 1.05x faster than the previous state-of-the-art model that used a similar architectural scheme. On the Penn Treebank dataset, our model can compose a novel recurrent cell that outperforms the widely-used LSTM cell, and other state-of-the-art baselines. Our cell achieves a test set perplexity of 62.4 on the Penn Treebank, which is 3.6 perplexity better than the previous state-of-the-art model. The cell can also be transferred to the character language modeling task on PTB and achieves a state-of-the-art perplexity of 1.214.

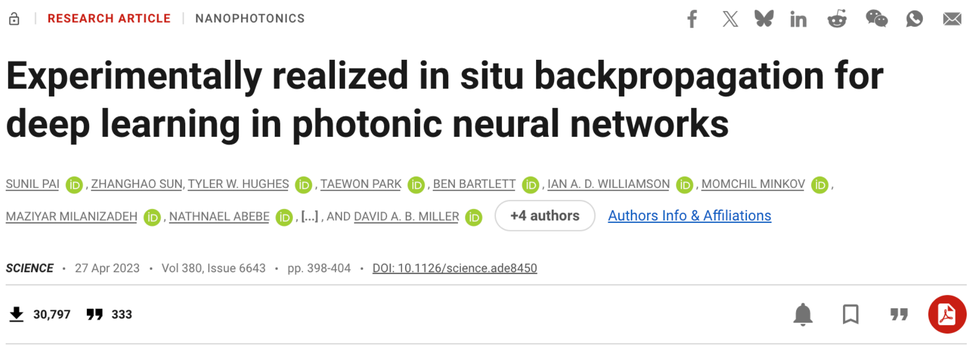
Conflicts: google.com

Community Implementations: 11 code implementations

Accelerate Training with Parallelism and Asynchronous Updates: In Neural Architecture Search, each gradient update to the controller parameters θ_c corresponds to training one child network to convergence. As training a child network can take hours, we use distributed training and asynchronous parameter updates in order to speed up the learning process of the controller (Dean et al., 2012). We use a parameter-server scheme where we have a parameter server of S shards, that store the shared parameters for K controller replicas. Each controller replica samples m different child architectures that are trained in parallel. The controller then collects gradients according to the results of that minibatch of m architectures at convergence and sends them to the parameter server in order to update the weights across all controller replicas. In our implementation, convergence of each child network is reached when its training exceeds a certain number of epochs. This scheme of parallelism is summarized in Figure 3.

When to stop an iterative algorithm?

Physical cost of sampling
or
gradient computation



RESEARCH ARTICLE | NANOPHOTONICS

Experimentally realized in situ backpropagation for deep learning in photonic neural networks

SUNIL PAI, ZHANGHAO SUN, TYLER W. HUGHES, TAEWON PARK, BEN BARTLETT, IAN A. D. WILLIAMSON, MOMCHIL MINKOV, MAZIYAR MILANIZADEH, NATHNAEL ABEBE, [...], AND DAVID A. B. MILLER +4 authors Authors Info & Affiliations

SCIENCE • 27 Apr 2023 • Vol 380, Issue 6643 • pp. 398-404 • DOI: 10.1126/science.ade8450

30,797 333

The speed of backpropagation was limited by the mechanics of the XY stage required to image spots throughout the chip, so our demonstration training experiments took up to 31 hours of real time to run, limited primarily by the wait time for the stage to settle on various groups of spots on the chip. Assuming T iterations, the stage needed to move a total of $15T$ times (5 for each of the three *in situ* backpropagation steps to be able to image all of the spots). For 1000 iterations, the stage needed to move a total of 15000 times which necessitated the need for automation for the stage of our proof-of-concept demonstration. In a final commercial im-

When to stop an iterative algorithm?

Physical cost of sampling
or
gradient computation

Reinvent 4: Modern AI–driven generative molecule design
HH Loeffler, J He, A Tibo, JP Janet, A Voronov… - Journal of …, 2024 - Springer
… To illustrate this, we consider a simple experiment where we start with the **REINVENT 4** prior: we run 500 epochs of RL with standard settings (σ = 128) and a scoring function that …
☆ Save 99 Cite Cited by 564 Related articles All 13 versions

Deepseekmath-v2: Towards self-verifiable mathematical reasoning
Z Shao, Y Luo, C Lu, ZZ Ren, J Hu, T Ye, Z Gou… - arXiv preprint arXiv …, 2025 - arxiv.org
Large language models have made significant progress in mathematical reasoning, which serves as an important testbed for AI and could impact scientific research if further advanced. …
☆ Save 99 Cite Cited by 98 Related articles All 4 versions »

Deepseekmath: Pushing the limits of mathematical reasoning in open language models
Z Shao, P Wang, Q Zhu, R Xu, J Song, X Bi… - arXiv preprint arXiv …, 2024 - arxiv.org
… To achieve this, we create the **DeepSeekMath** Corpus, a large-scale high-quality pre-training corpus comprising 120B math tokens. This dataset is extracted from the Common Crawl (…
☆ Save 99 Cite Cited by 9722 Related articles All 12 versions »

When to stop an iterative algorithm?

Physical cost of sampling
or
gradient computation

Require a sharp envelope
(potentially allowing for slower envelope construction)

When to stop an iterative algorithm?

Physical cost of sampling

↓

Require a sharp envelope
(potentially allowing for slower envelope construction)

↑

Other data-dependent stopping time,
e.g. $\min\{k : \ \|x_t - x_{t-1}\| < \varepsilon\}$

When to stop an iterative algorithm?

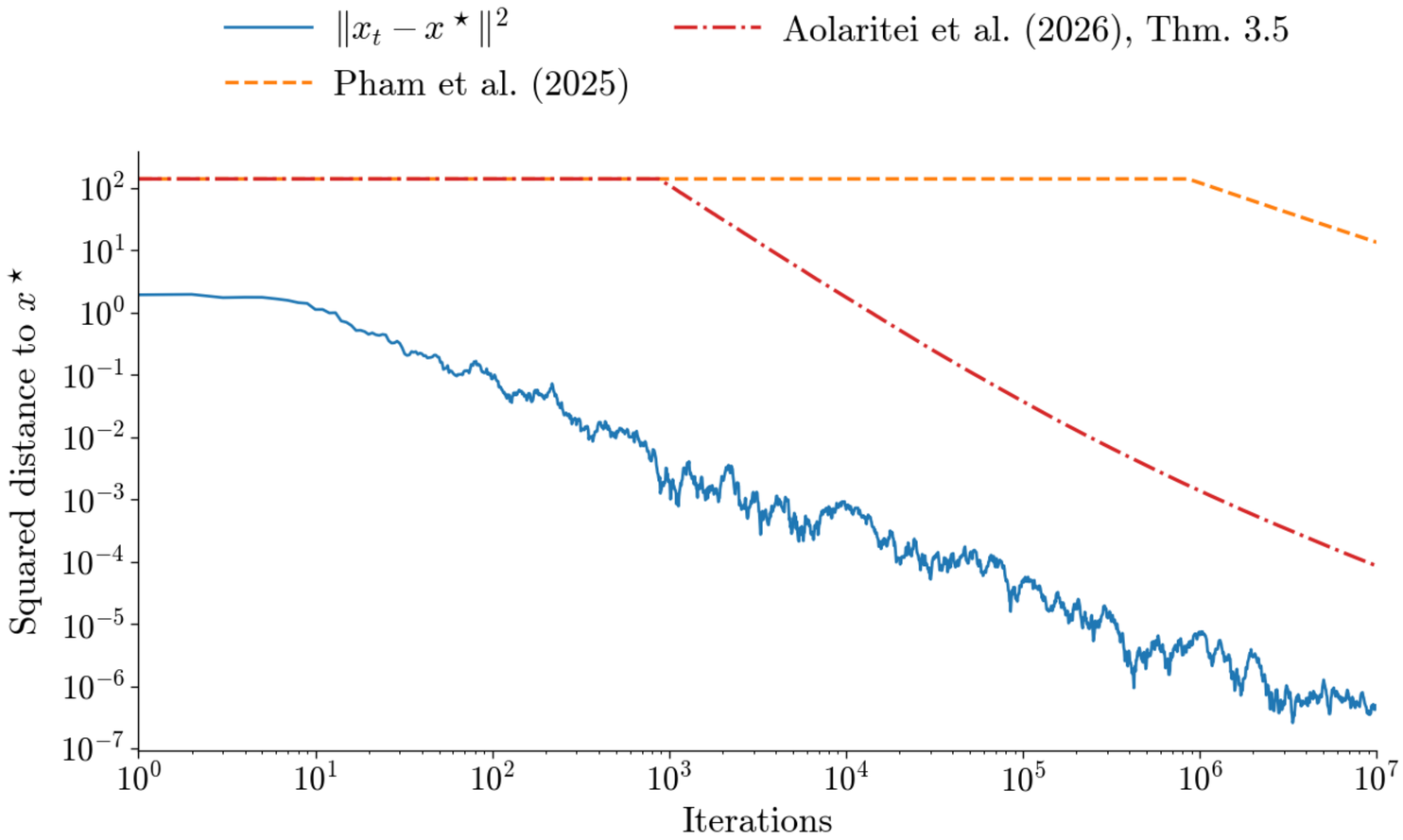
Physical cost of sampling

↓

Require a sharp envelope

↑

Other data-dependent stopping time,
e.g. $\min\{k : \|x_t - x_{t-1}\| < \varepsilon\}$



Envelopes in the literature for a ridge-regression example

Desiderata 2

Generalizability (for different SGD assumptions)

	Performance measure				
Assmp.	$\|\|x_t - x^\star\|\|^2$	$\|\|\nabla F(x_t)\|\|$	wt. avg. $F(x_s) - F^\star$	wt. avg. $\|\|\nabla F(x_s)\|\|^2$	…
Strongly convex	Pham et al., 2025 Aolaritei et al., 2026				
Convex			Aolaritei et al., 2025		
Smooth, non-convex				Aolaritei et al., 2025	
Local self-concordance					
Linear growth in gradients					
Polyak–Łojasiewicz					
…					

Desiderata 2

Generalizability (for different SGD assumptions)

	Performance measure				
Assmp.	$\|\|x_t - x^\star\|\|^2$	$\|\|\nabla F(x_t)\|\|$	wt. avg. $F(x_s) - F^\star$	wt. avg. $\|\|\nabla F(x_s)\|\|^2$	…
Strongly convex	Pham et al., 2025 Aolaritei et al., 2026	✓	✓	✓	
Convex	✓	✓	Aolaritei et al., 2025	✓	
Smooth, non-convex		✓	✓	Aolaritei et al., 2025	
Local self-concordance	✓	✓	✓	✓	
Linear growth in gradients		✓	✓		
Polyak–Łojasiewicz	✓	✓	✓	✓	

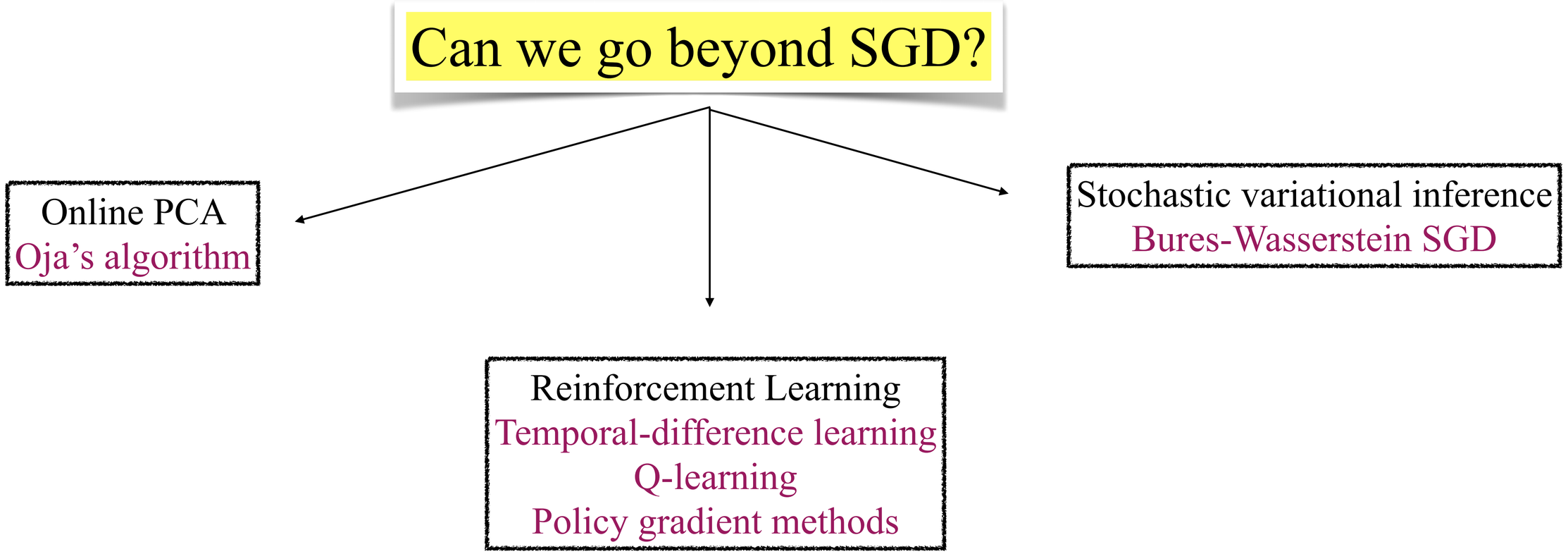
…

Desiderata 2

Generalizability (for different SGD assumptions)

Do we have to go case-by-case?

Generalizability



Progress so far

- [x] Why envelopes, and must we build them case by case?
- [ ] So what's new?- a unified perspective.
- [ ] How to construct sharper envelopes?
- [ ] Application

Recall μ-strongly convex SGD (from last week)

- **Performance measure:** $Z_t := \|e_t\|^2,\ e_t := x_t - x^\star$.

$x^\star$: global minima

- **Line of attack**

$$Z_t \leq \underbrace{(1 - 2\mu\eta_t)}_{\text{effect of previous step's error}} Z_{t-1} + \underbrace{\eta_t^2 \|g_t\|^2}_{\text{effect of current stochastic } \textit{gradient}} + \underbrace{2\eta_t \langle e_{t-1}, \xi_t \rangle}_{\text{unobserved noise}}, \quad \xi_t = \nabla F(x_{t-1}) - g_t.$$

μ-strongly convex: $\langle \nabla F(x) - \nabla F(y), x - y \rangle \geq \mu \|x - y\|^2$

Projected SGD: $x_t = \Pi_{\mathcal{X}}(x_{t-1} - \eta_t g_t)$

Recall μ-strongly convex SGD (from last week)

- **Performance measure:** $Z_t := \|e_t\|^2,\ e_t := x_t - x^\star$.

$x^\star$: global minima

- **Line of attack**

$$Z_t \le \underbrace{(1-2\mu\eta_t)}_{a_t} Z_{t-1} + \underbrace{\eta_t^2\|g_t\|^2}_{b_t} + \underbrace{2\eta_t\langle e_{t-1}, \xi_t\rangle}_{\langle v_t, \xi_t\rangle \text{ (noise)}}$$

Assume sub-Gaussian ξ_t, or $\|g_t\| \le c$

↓

Empirical Bernstein envelope on the sum $\sum_{s=1}^{t} \eta_s\langle e_{s-1}, \xi_s\rangle$ via sub-Gamma stitching

μ-strongly convex: $\langle \nabla F(x) - \nabla F(y), x - y\rangle \ge \mu\|x-y\|^2$

Projected SGD: $x_t = \Pi_{\mathcal{X}}(x_{t-1} - \eta_t g_t)$

Recall μ-strongly convex SGD (from last week)

- **Performance measure:** $Z_t := \|e_t\|^2,\ e_t := x_t - x^\star$.

$x^\star$: global minima

- **Line of attack**

$$Z_t \le \underbrace{(1-2\mu\eta_t)}_{a_t} Z_{t-1} + \underbrace{\eta_t^2\|g_t\|^2}_{b_t} + \underbrace{2\eta_t\langle e_{t-1}, \xi_t\rangle}_{\langle v_t, \xi_t\rangle \text{ (noise)}}$$

Assume sub-ψ condition on ξ_t, or $\|g_t\| \le c$.

↓

Empirical Bernstein envelope on the sum $\sum_{s=1}^{t} \eta_s\langle e_{s-1}, \xi_s\rangle$ via sub-Gamma stitching

μ-strongly convex: $\langle \nabla F(x) - \nabla F(y), x - y\rangle \ge \mu\|x-y\|^2$

Projected SGD: $x_t = \Pi_{\mathcal{X}}(x_{t-1} - \eta_t g_t)$

Recall μ-strongly convex SGD (from last week)

- **Performance measure:** $Z_t := \|e_t\|^2,\ e_t := x_t - x^\star$.

$x^\star$: global minima

- **Line of attack**

$$Z_t \le \underbrace{(1-2\mu\eta_t)}_{a_t} Z_{t-1} + \underbrace{\eta_t^2\|g_t\|^2}_{b_t} + \underbrace{2\eta_t\langle e_{t-1}, \xi_t\rangle}_{\langle v_t\ ,\xi_t\rangle}$$

Assume (a stronger version) of sub-ψ condition on ξ_t, or $\|g_t\| \le c$.

$\downarrow$

Anytime-valid envelope on $\sum_{s=1}^{t} \langle v_s, \xi_s\rangle$.

μ-strongly convex: $\langle \nabla F(x) - \nabla F(y), x - y\rangle \ge \mu\|x-y\|^2$

Projected SGD: $x_t = \Pi_{\mathcal{X}}(x_{t-1} - \eta_t g_t)$

A general line of attack

- Derive recursion: $Z_t \leq a_t Z_{t-1} + b_t + \langle v_t \ , \xi_t \rangle$ (from some starting assumptions) .

  where a_t, v_t are *predictable* (depends on past, potentially unobserved data),
  and b_t is *observable*.

- Problem-specific *optimization module.*

- Can be derived independently for SGD with diff. asmp, for diff. Measures.

- Step-sizes/learning rates only need to be predictable.

A general line of attack

- Derive recursion: $Z_t \leq a_t Z_{t-1} + b_t + \langle v_t \ , \xi_t \rangle$ (from some starting assumptions) .

  where a_t, v_t are *predictable* (depends on past, potentially unobserved data),
  and b_t is *observable*.

- Problem-specific *optimization module.*

- Can be derived for other applications eg. Q-learning, Oja's algorithm, etc.

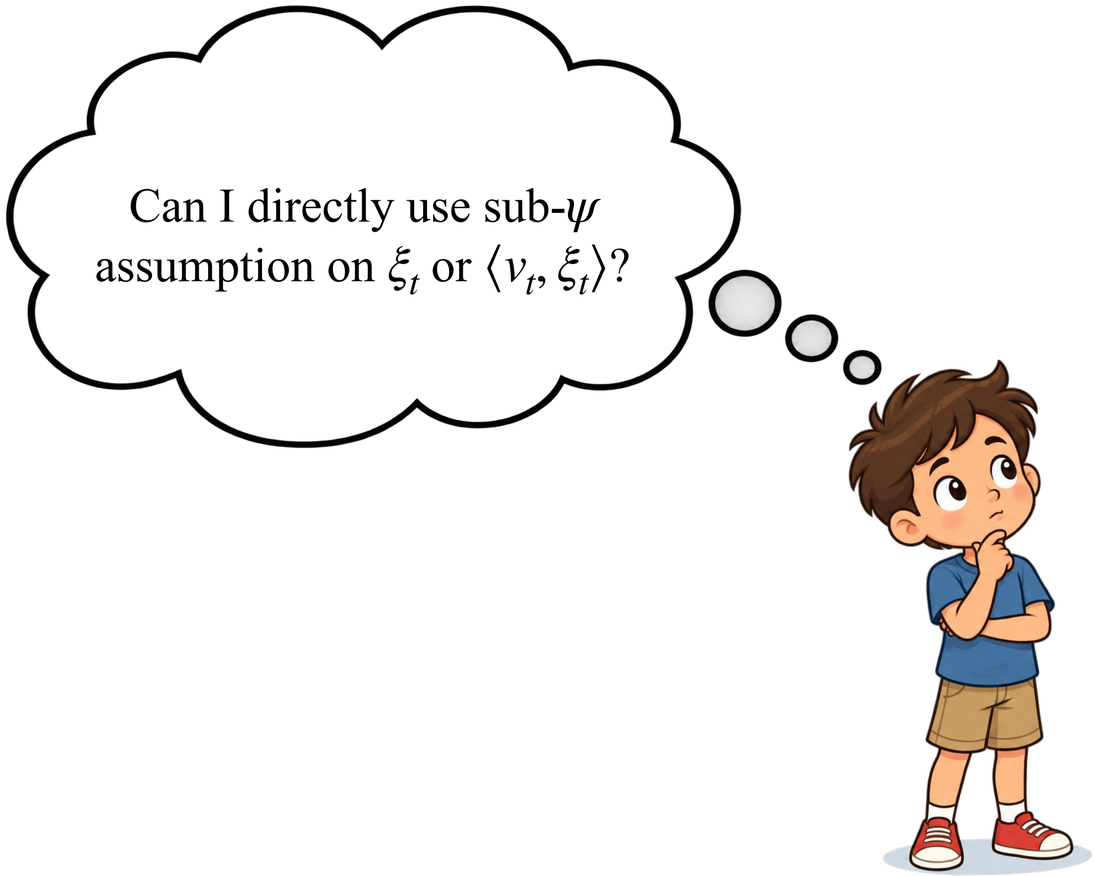
- Construct anytime-valid boundaries for $\sum_{s=1}^{t} \langle v_s, \xi_s \rangle$.

Progress So far

- [x] Why envelopes, and must we build them case by case?
- [x] So what's new?- a unified perspective.
- [ ] How to construct sharper envelopes?
- [ ] Application

Construct anytime-valid boundaries for $\sum_{s=1}^{t} \langle v_s, \xi_s \rangle$

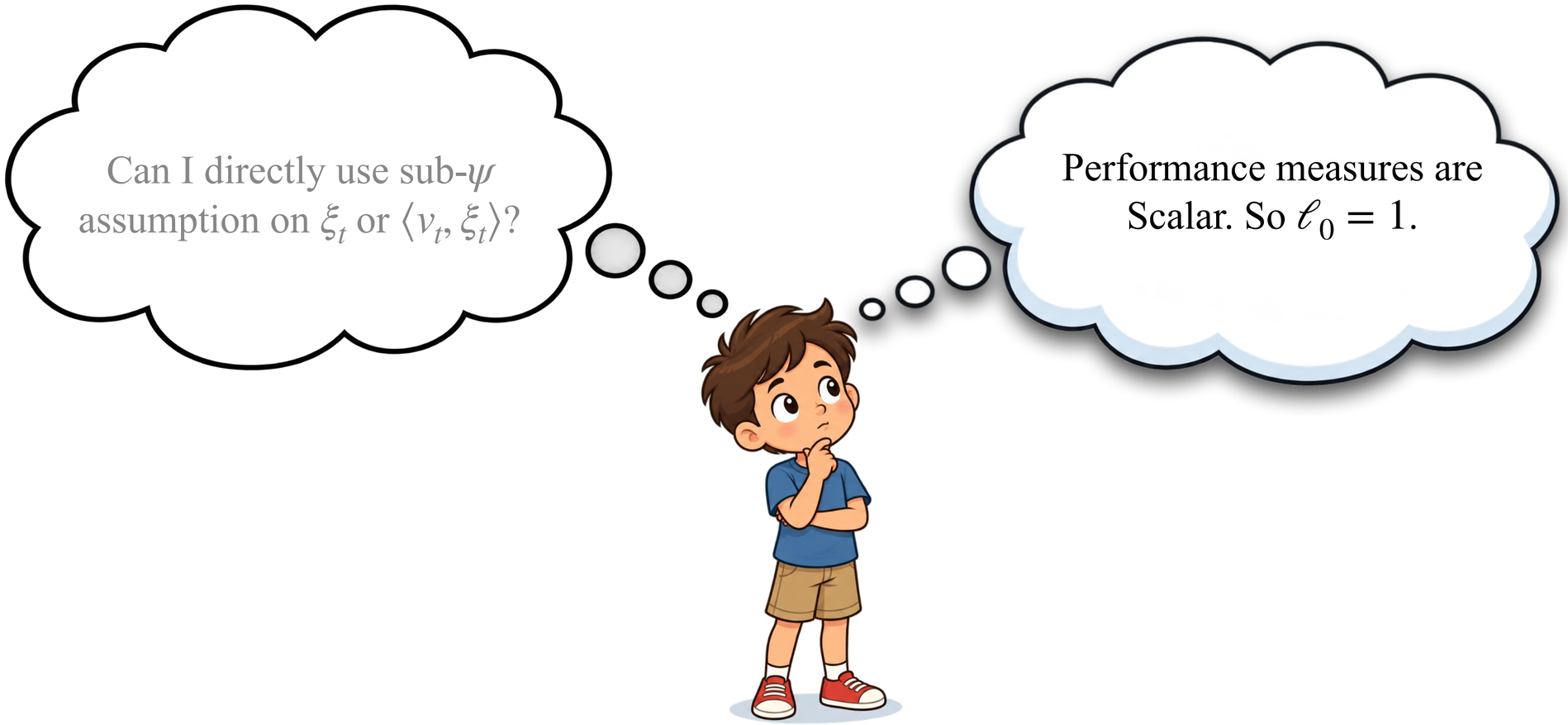
- Let $\psi : [0, \lambda_{\max}) \to \mathbb{R}_+$ be a *CGF-like*[1] function.



[1] strongly convex, $\psi(0) = \psi'(0) = 0$, $\psi''(0) > 0$.

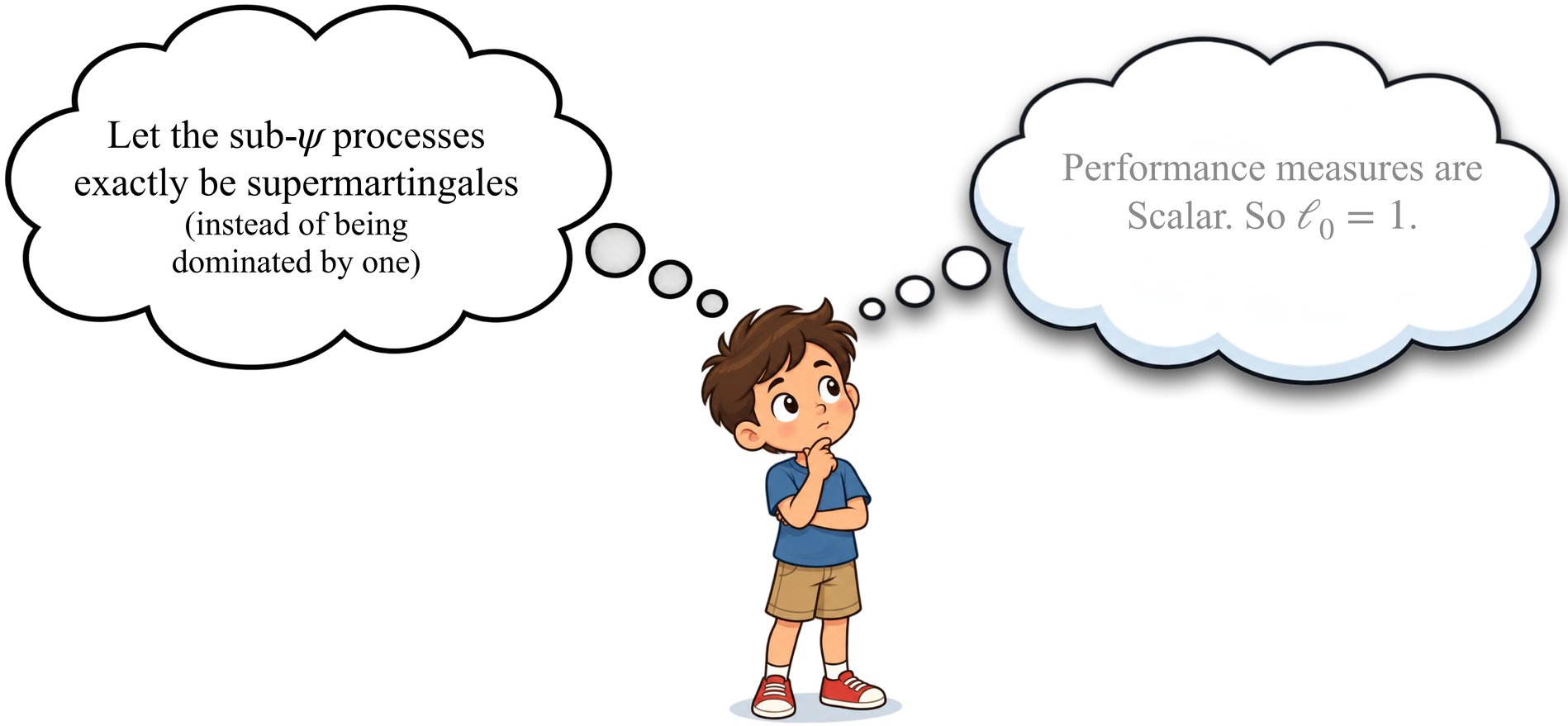
Construct anytime-valid boundaries for $\sum_{s=1}^{t} \langle v_s, \xi_s \rangle$

- Let $\psi : [0, \lambda_{\max}) \to \mathbb{R}_+$ be a *CGF-like* function.



Construct anytime-valid boundaries for $\sum_{s=1}^{t} \langle v_s, \xi_s \rangle$

- Let $\psi : [0, \lambda_{\max}) \to \mathbb{R}_+$ be a *CGF-like* function.



Construct anytime-valid boundaries for $\sum_{s=1}^{t} \langle v_s, \xi_s \rangle$

- Let $\psi : [0, \lambda_{\max}) \to \mathbb{R}_+$ be a *CGF-like* function.

sub-ψ condition
(Howard et al. 2020, Whitehouse et al. 2026)

For $\lambda \in [0, \lambda_{\max})$, $u \in \mathbb{S}^{d-1}$, $\exp\left(\sum_{s=1}^{t} \left[\lambda \langle u, \xi_s \rangle - \psi(\lambda)\, \sigma_s^2(u) \right] \right) \leq$ supermartingale.

$\sigma_s^2(v) := v^\top \Delta V_s \, v / \|v\|^2$

ΔV_s: variance process

Construct anytime-valid boundaries for $\sum_{s=1}^{t} \langle v_s, \xi_s \rangle$

- Let $\psi : [0, \lambda_{\max}) \to \mathbb{R}_+$ be a *CGF-like* function.

(Mild strengthening of) sub-ψ condition

(Bonnerjee and Ramdas, 2026)

For $\lambda \in [0, \lambda_{\max})$, $u \in \mathbb{S}^{d-1}$, $\exp\left(\sum_{s=1}^{t} \left[\lambda \langle u, \xi_s \rangle - \psi(\lambda)\, \sigma_s^2(u) \right] \right) =$ supermartingale.



$\sigma_s^2(v) := v^\top \Delta V_s \, v / \|v\|^2$

ΔV_s: variance process

Stitching boundary for $\xi_t = g_t - \mathbb{E}_{t-1} g_t,\ \|g_t\| \le c$

- $(\xi_t, g_t g_t^\top)$ is sub-$\psi_{E,c}$ process.
- Therefore, sub-exponential stitching can be done.

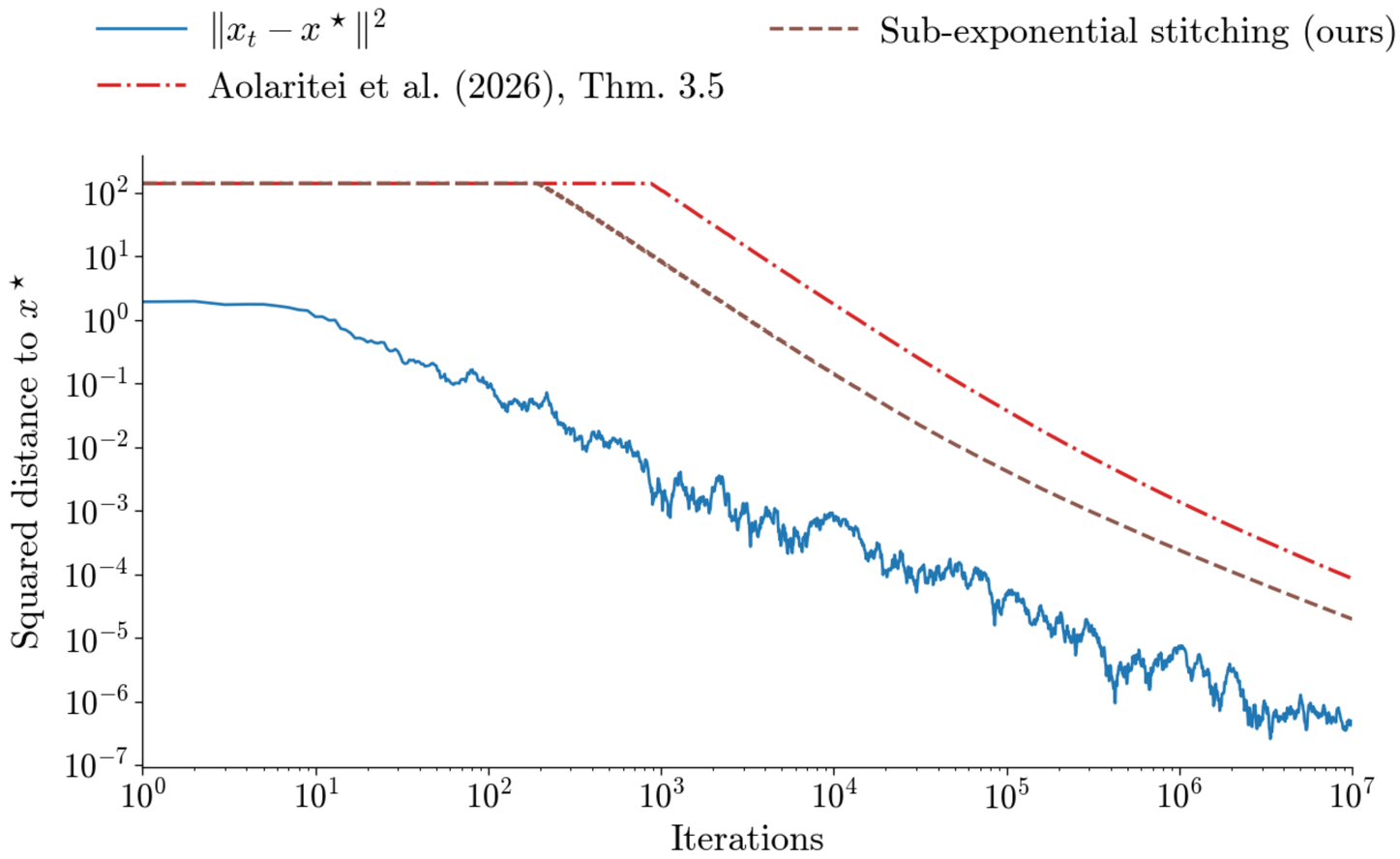
E: exponential; *G*: gamma

Stitching boundary for $\xi_t = g_t - \mathbb{E}_{t-1} g_t,\ \|g_t\| \le c$

- $(\xi_t, g_t g_t^\top)$ is sub-$\psi_{E,c}$ process.
- Therefore, sub-exponential stitching can be done.
- $\psi_{E,c} \le \psi_{G,c}$ (Howard et al. (2020), Table 5, (7)):
  a sub-Gamma stitching can be done, recovering Aolaritei et al. (2026).

E: exponential; *G*: gamma

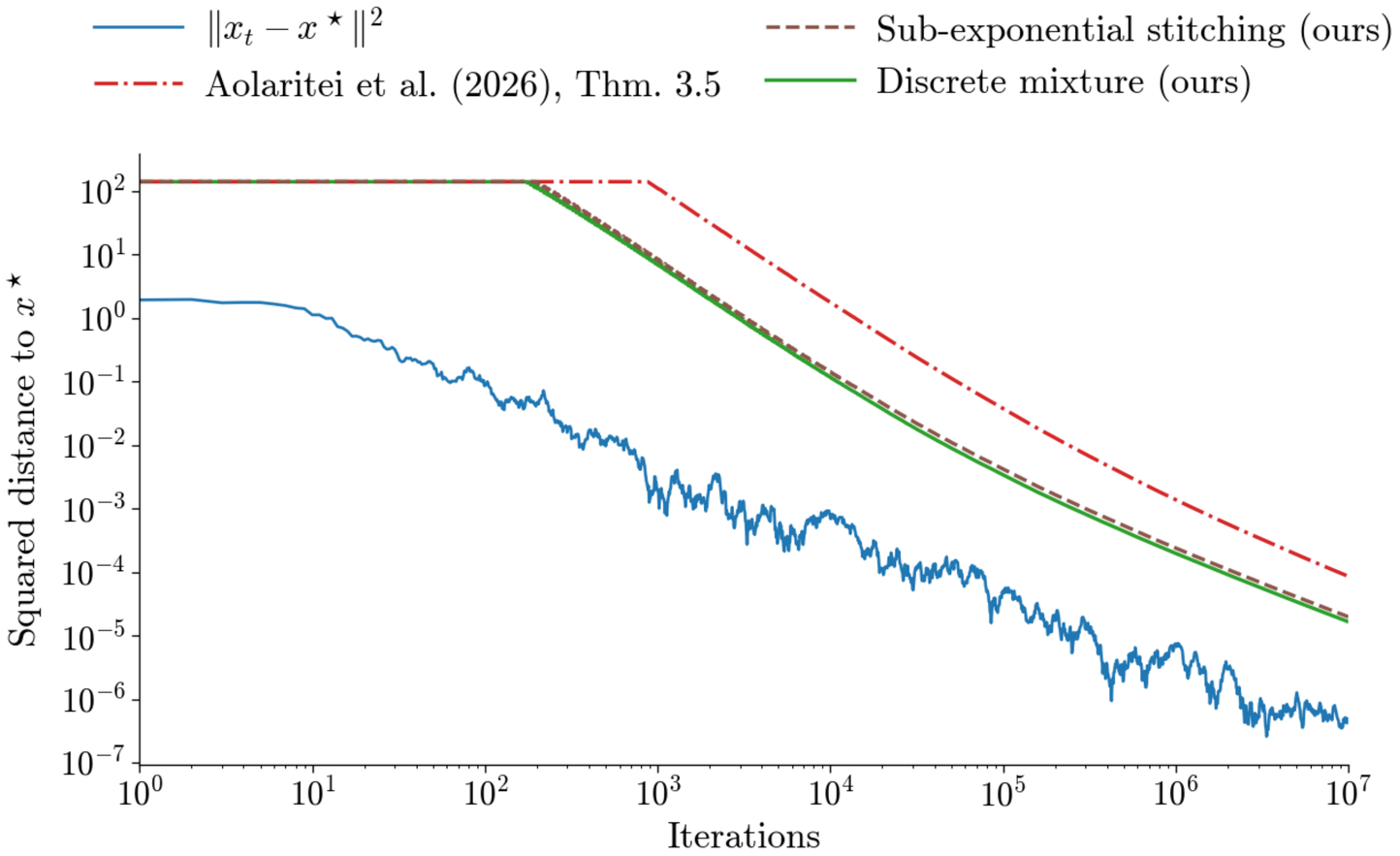
Stitching boundary for $\xi_t = g_t - \mathbb{E}_{t-1} g_t,\ \|g_t\| \le c$



Benefit from $\psi_{E,c} \le \psi_{G,c}$.

E: exponential; G: gamma

Stitching boundary for $\xi_t = g_t - \mathbb{E}_{t-1} g_t,\ \|g_t\| \le c$



Benefit from $\psi_{E,c} \le \psi_{G,c}$. A slightly sharper boundary can be obtained via Discrete mixture.

E: exponential; *G*: gamma

Betting martingale for $\xi_t = g_t - \mathbb{E}_{t-1} g_t,\ \|g_t\| \le c$

- At round s, gambler stakes wealth fraction λ_s against $\mathbb{E}_{t-1}\langle v_s, \xi_s\rangle = 0$.

Notation

$X_t = \langle v_t, g_t\rangle$

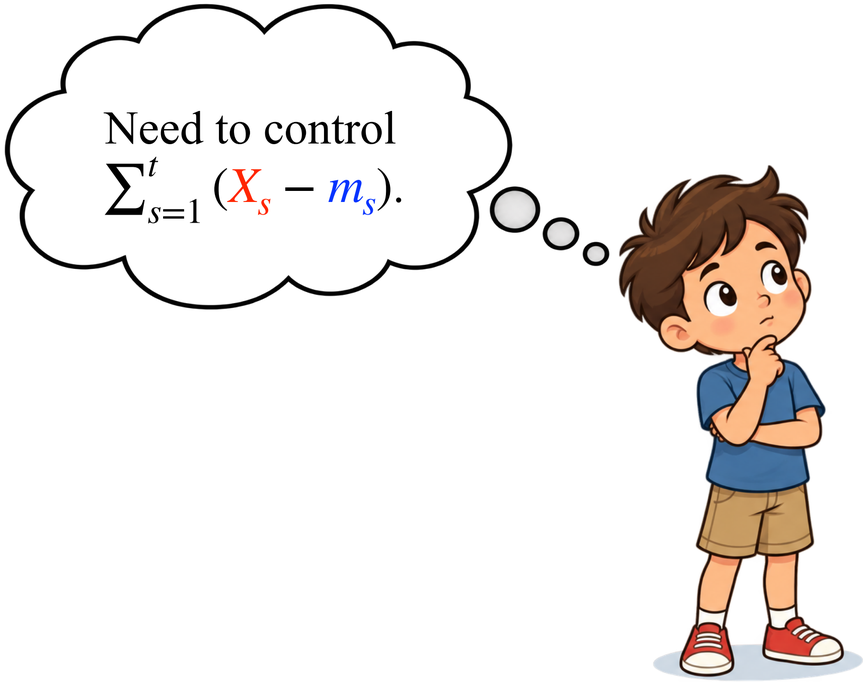
$m_t = \langle v_t, \mathbb{E}_{t-1} g_t\rangle$.

Betting martingale for $\xi_t = g_t - \mathbb{E}_{t-1} g_t,\ \|g_t\| \le c$

- At round s, gambler stakes wealth fraction λ_s against $\mathbb{E}_{s-1}(X_s - m_s) = 0$.

Notation
$X_t = \langle v_t, g_t \rangle$
$m_t = \langle v_t, \mathbb{E}_{t-1} g_t \rangle$.

- Wealth process: $K_t = \prod_{1 \le s \le t} (1 + \lambda_s (X_s - m_s)),\ K_0 = 1.$



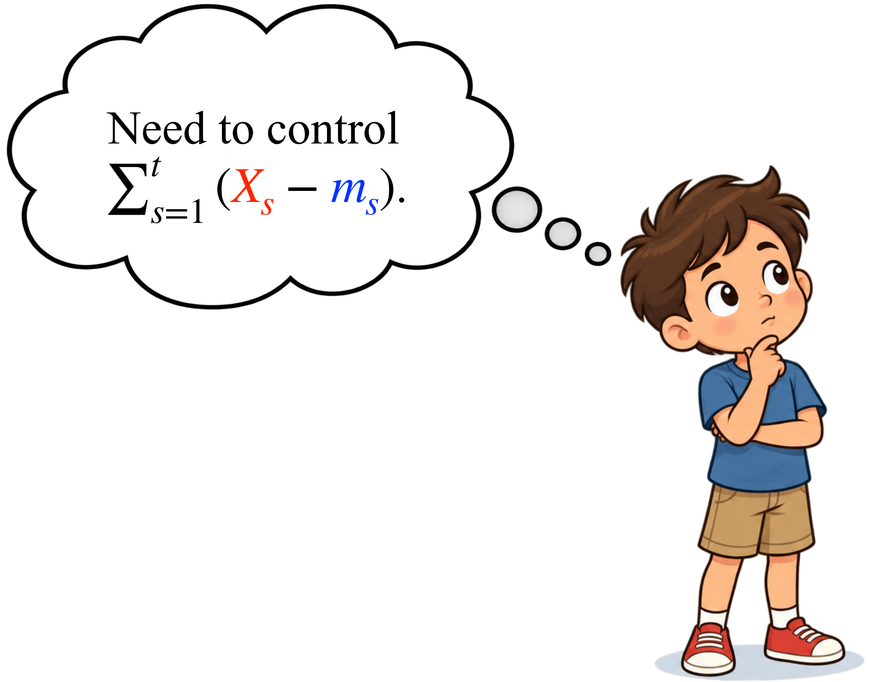
- Taking log boils down to controlling $\sum_{s=1}^{t} \lambda_s (X_s - m_s)$.
- Selecting $\lambda_1 = \ldots = \lambda_s = \lambda$ does not exploit adaptive betting.

Betting martingale for $\xi_t = g_t - \mathbb{E}_{t-1} g_t,\ \|g_t\| \le c$

- At round s, gambler stakes wealth fraction λ_s against $\mathbb{E}_{s-1}(X_s - m_s) = 0$.

Notation
$X_t = \langle v_t, g_t \rangle$
$m_t = \langle v_t, \mathbb{E}_{t-1} g_t \rangle$.

- Wealth process: $K_t = \prod_{\substack{1 \le s \le t \\ \lambda_s \|v_s\| < 1/c}} (1 + \lambda_s (X_s - m_s)),\ K_0 = 1.$



- We would like to exploit $\|\xi_t\| \le c + \|\mathbb{E}_{t-1} g_t\|$, which means

$$\lambda_s \le 1/(c\|v_s\| + m_s).$$

- Bet more iff $m_s < 0$. O.w., bet less.

Betting martingale for $\xi_t = g_t - \mathbb{E}_{t-1} g_t,\ \|g_t\| \le c$

- At round s, gambler stakes wealth fraction λ_s against $\mathbb{E}_{s-1}(X_s - m_s) = 0$.

Notation
$X_t = \langle v_t, g_t \rangle$
$m_t = \langle v_t, \mathbb{E}_{t-1} g_t \rangle$.

- Choose $\boxed{\lambda_s = \dfrac{\lambda}{1 + \lambda m_s}}$ for some $\lambda > 0$.

- Wealth process: $\widetilde{K}_t = \prod_{s=1}^{t} (1 + \lambda_s (X_s - m_s)) = \prod_{s=1}^{t} \dfrac{1 + \lambda X_s}{1 + \lambda m_s}$.

- Bet more iff $m_s < 0$. O.w., bet less.

Betting boundary for $\xi_t = g_t - \mathbb{E}_{t-1} g_t,\ \|g_t\| \le c$

Notation
$X_t = \langle v_t, g_t \rangle$
$m_t = \langle v_t, \mathbb{E}_{t-1} g_t \rangle$.

- At round s, gambler stakes wealth fraction λ_s against $\mathbb{E}_{s-1}(X_s - m_s) = 0$.
- Choose $\boxed{\lambda_s = \dfrac{\lambda}{1 + \lambda m_s}}$ for some $\lambda > 0$.
- Wealth process: $\widetilde{K}_t = \prod_{s=1}^{t} (1 + \lambda_s (X_s - m_s)) = \prod_{s=1}^{t} \dfrac{1 + \lambda X_s}{1 + \lambda m_s}$.
- $\log \widetilde{K}_t = \lambda \sum_{s=1}^{t} (X_s - m_s) - \mathscr{B}(\lambda)$, $\boxed{\mathscr{B}(\lambda) := \sum_{s=1}^{t} \psi_{E,1}(-\lambda X_s) - \sum_{s=1}^{t} \psi_{E,1}(-\lambda m_s)}$
- Envelope: $\sup\{x : \exp(\lambda x - \mathscr{B}(\lambda)) < 1/\alpha\}$

Betting boundary for $\xi_t = g_t - \mathbb{E}_{t-1} g_t,\ \|g_t\| \le c$

Notation
$X_t = \langle v_t, g_t \rangle$
$m_t = \langle v_t, \mathbb{E}_{t-1} g_t \rangle$.

- At round s, gambler stakes wealth fraction λ_s against $\mathbb{E}_{s-1}(X_s - m_s) = 0$.
- Choose $\boxed{\lambda_s = \dfrac{\lambda}{1 + \lambda m_s}}$ for some $\lambda > 0$.
- Wealth process: $\widetilde{K}_t = \prod_{s=1}^{t} (1 + \lambda_s (X_s - m_s)) = \prod_{s=1}^{t} \dfrac{1 + \lambda X_s}{1 + \lambda m_s}$.
- $\log \widetilde{K}_t = \lambda \sum_{s=1}^{t} (X_s - m_s) - \mathscr{B}(\lambda)$, $\boxed{\mathscr{B}(\lambda) := \sum_{s=1}^{t} \psi_{E,1}(-\lambda X_s) - \sum_{s=1}^{t} \psi_{E,1}(-\lambda m_s)}$
- Still depends on the choice of λ: a discrete mixture over λ.

Sharpness of betting boundary when $\|g_t\| \le c$

Fixed λ envelope: $(\mathscr{B}(\lambda) + \log(1/\alpha))/\lambda$, where,

$$\mathscr{B}(\lambda) := \sum_{s=1}^{t} \psi_{E,1}(-\lambda X_s) - \sum_{s=1}^{t} \psi_{E,1}(-\lambda m_s)$$

Notation

$X_t = \langle v_t, g_t \rangle$

$m_t = \langle v_t, \mathbb{E}_{t-1} g_t \rangle$.

Envelope with fixed λ (before stitching or mixing)

$$(\psi(\lambda\|v_s\|)\|g_s\|^2 + \log(1/\alpha))/\lambda$$

Sharpness of betting boundary when $\|g_t\| \le c$

Fixed λ envelope: $(\mathscr{B}(\lambda) + \log(1/\alpha))/\lambda$, where,

$$\mathscr{B}(\lambda) := \sum_{s=1}^{t} \psi_{E,1}(-\lambda X_s) - \underbrace{\sum_{s=1}^{t} \psi_{E,1}(-\lambda m_s)}_{\ge 0}$$

Notation

$X_t = \langle v_t, g_t \rangle$

$m_t = \langle v_t, \mathbb{E}_{t-1} g_t \rangle$.

Envelope with fixed λ (before stitching or mixing)

$(\psi(\lambda\|v_s\|)\|g_s\|^2 + \log(1/\alpha))/\lambda$

Three sources of slackness

- Source 0 : $\underbrace{\sum_{s=1}^{t} \psi_{E,1}(-\lambda m_s)}_{\geq 0}$ (minor slackness, since m_s is small for large iterates).

Notation
$X_t = \langle v_t, g_t \rangle$
$m_t = \langle v_t, \mathbb{E}_{t-1} g_t \rangle$.

- $\psi_{E,1}(-\lambda X_s) \approx \sum_{j=2}^{\infty} \frac{\lambda^j \|X_s\|^j}{j} = \sum_{j=2}^{\infty} \frac{\lambda^j \|v_s\|^j \|g_s\|^j}{j}$

$\leq$ (Source 1) $\sum_{j=2}^{\infty} \frac{\lambda^j \|v_s\|^j c^{j-2} \|g_s\|^2}{j} = \underbrace{\psi_{E,c}(\lambda \|v_s\|) \|g_s\|^2}_{\text{sub-exponential}}$

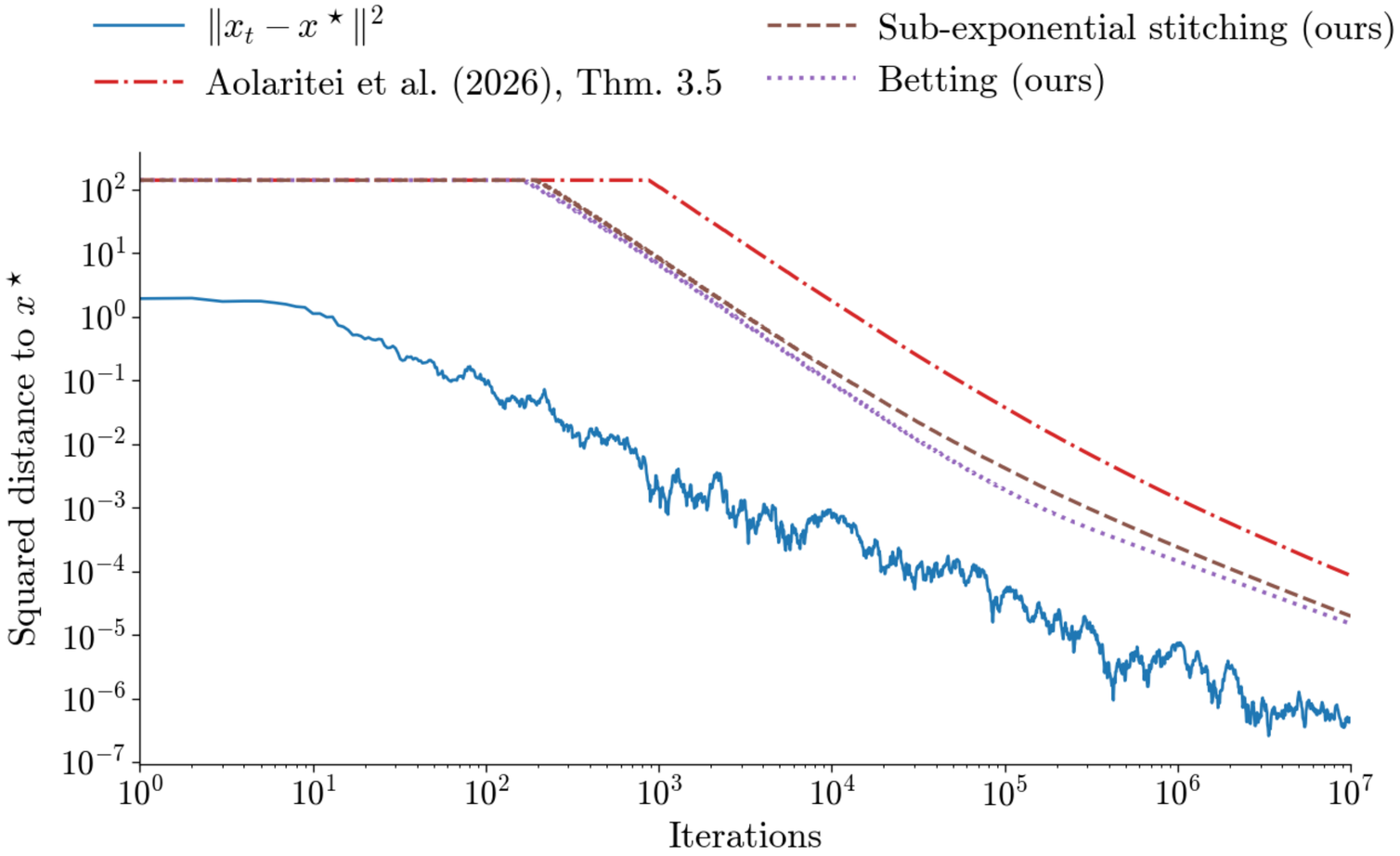
Envelope with fixed λ (before stitching or mixing)
$(\psi(\lambda \|v_s\|) \|g_s\|^2 + \log(1/\alpha))/\lambda$

$\leq$ (Source 2) $\sum_{j=2}^{\infty} \frac{\lambda^j \|v_s\|^j c^{j-2} \|g_s\|^2}{2} = \underbrace{\psi_{G,c}(\lambda \|v_s\|) \|g_s\|^2}_{\text{sub-gamma}}$

Betting $\leq$ sub-exp stitching $\leq$ sub-gamma stitching.

Betting boundary is sharpest

Betting ≤ sub-exp stitching ≤ sub-gamma stitching.



Betting boundary is sharpest

	#iterations for envelopes to reach 0.01	Wall-clock time (s)
Aolaritei et. al. (2026)	238,446	0.018702
Sub-exp stitching	53,545	0.016587
Betting	33,773	0.214555

Ridge regression example. Details in §5.1, Bonnerjee & Ramdas (2026).

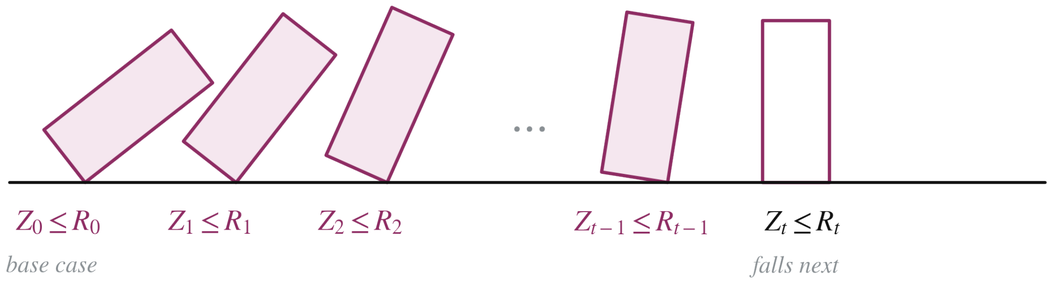
Unifying the two modules

Constructing a recursion for performance measure Z_t

(Optimization module)

Constructing an anytime-valid envelope for $\sum_{s=1}^{t} \langle \xi_t, v_t \rangle$

(Statistical module)



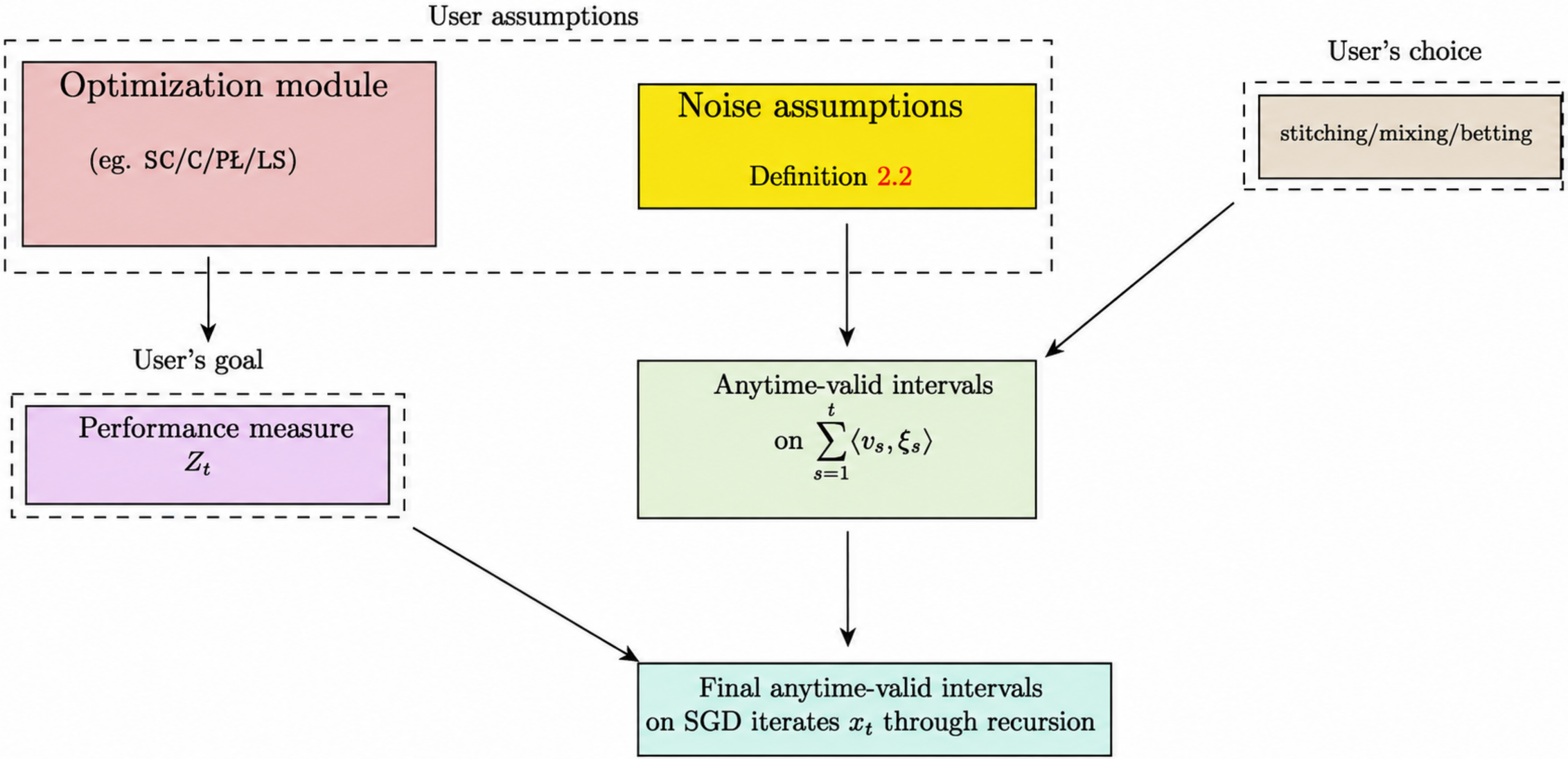
Replace any unobservable quantity by

- either, an observable upper bound
- or, by the corresponding quantity from the envelope.

(i.e., replace Z_{t-1} by the corresponding envelope R_{t-1} recursively)

Induction ensures validity!

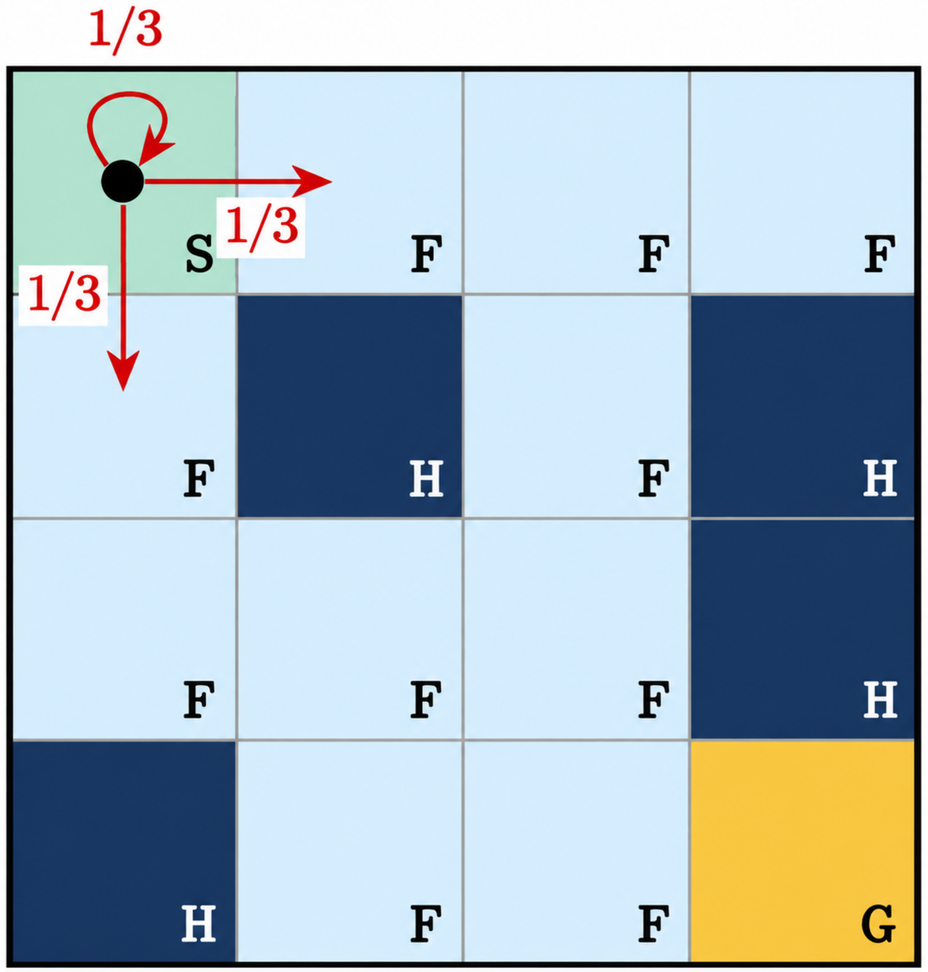
A complete workflow



Progress so far

- [x] Why envelopes, and must we build them case by case?
- [x] So what's new?- a unified perspective.
- [x] How to construct sharper envelopes?
- [ ] Application

Application: TD-learning on FrozenLake[1]

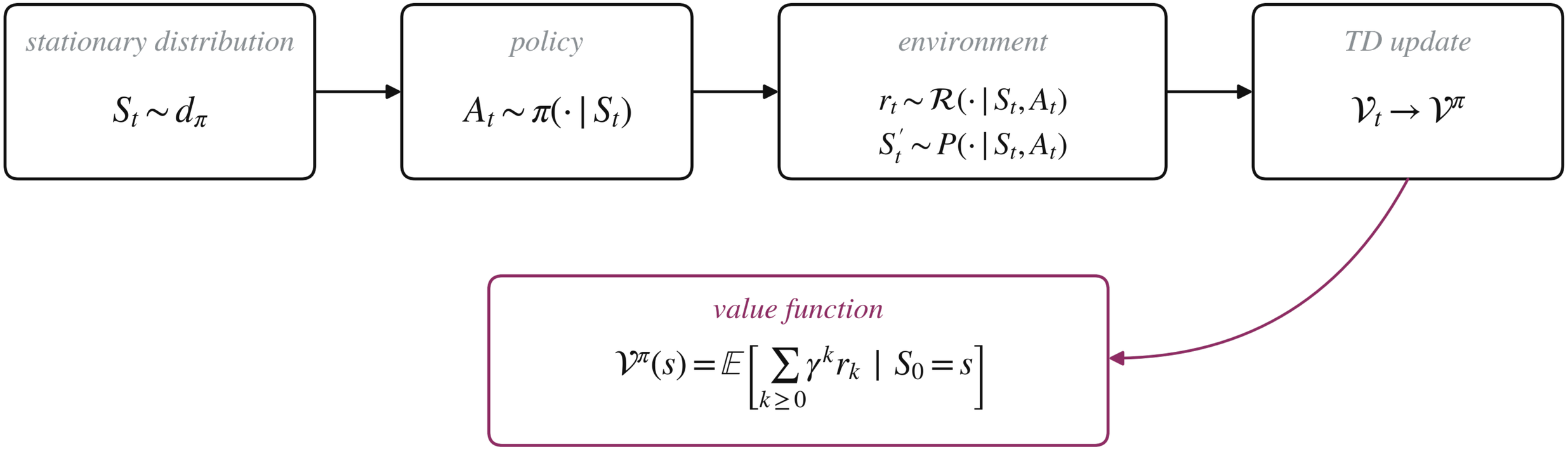


- Start *S*, Frozen tiles *F*, Holes *H*, Goal *G*.
- the agent moves at intended direction, or in either perpendicular direction, each w.p. 1/3.

[1] https://gymnasium.farama.org/environments/toy_text/frozen_lake

Application: TD-learning on FrozenLake

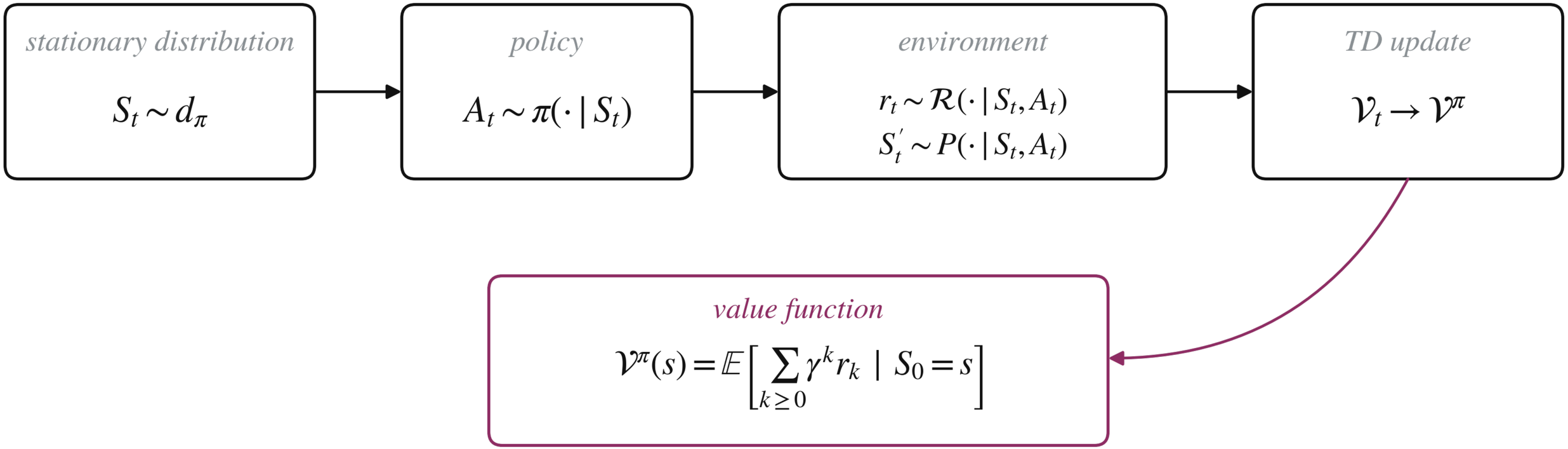
Goal: evaluate some policy[1] π.



[1]policy: action probabilities $\pi(a \mid s)$

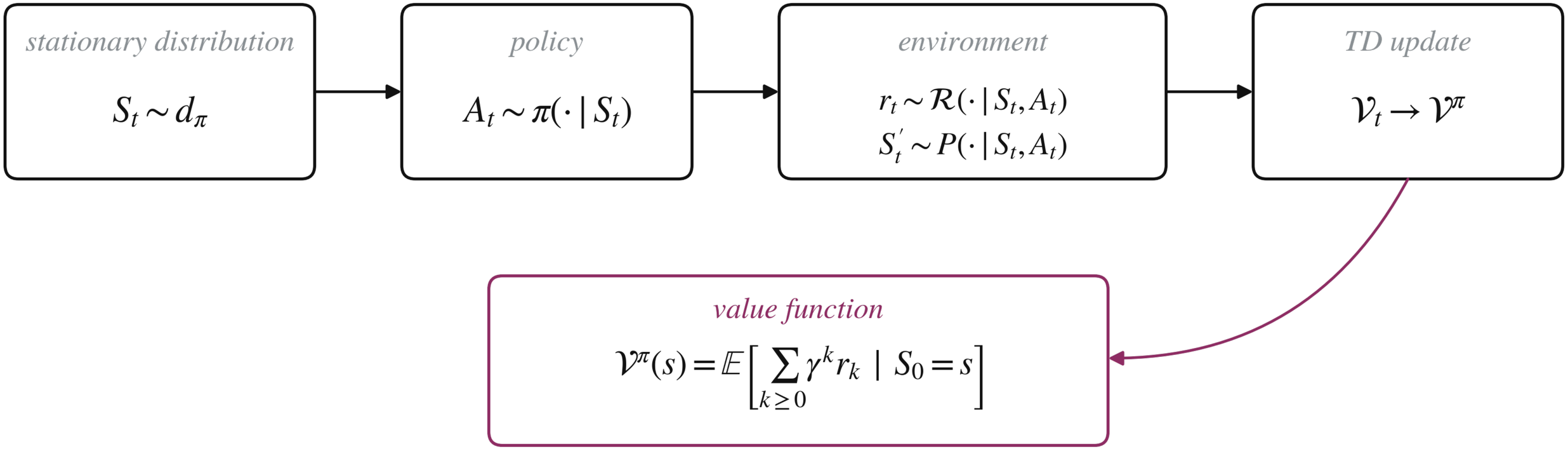
Application: TD-learning on FrozenLake

TD approximates $\mathscr{V}^{\pi}(s)$ by $\mathscr{V}_{\theta}(s) = \phi(s)^{\top}\theta$.

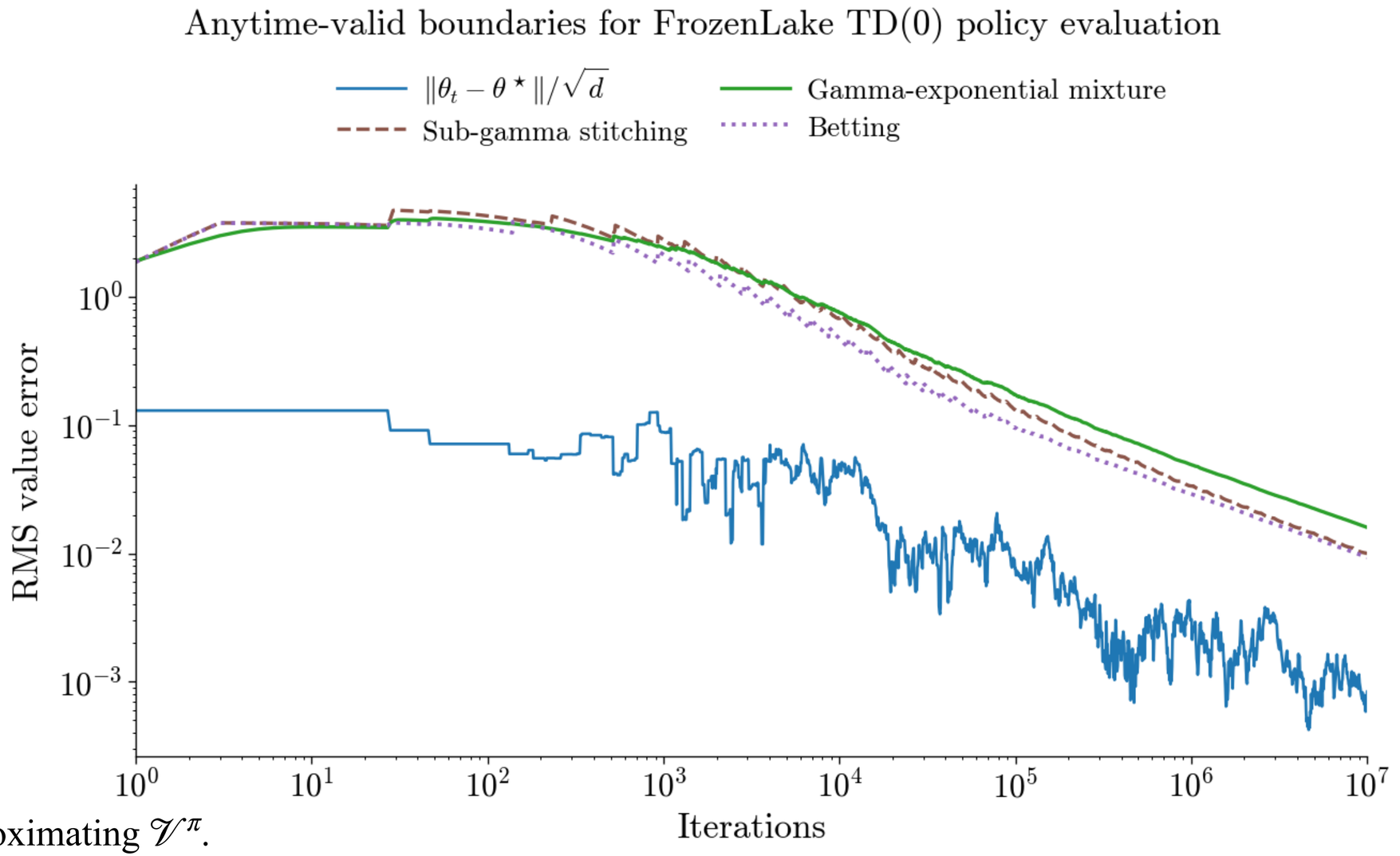


Application: TD-learning on FrozenLake

TD learns θ_t iteratively. When to stop a TD-learning algorithm?



Application: TD-learning on FrozenLake



$\theta^\star$: optimal θ approximating $\mathscr{V}^\pi$.

Paper

Bonnerjee, S., & Ramdas, A. (2026). A general recipe for data-adaptive anytime-valid envelopes for iterative algorithms. arXiv:2610.23570.

Questions?

